

JAVA INTERVIEW PREPARATION

CHAPTER 25

COMPLETABLEFUTURE AND ASYNCHRONOUS COMPOSITION

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

CompletableFuture and Asynchronous Composition: Building Non-Blocking Dependency Graphs

Senior / Lead Java Developer Reading Chapter

CompletableFuture represents a value and a graph of actions that should run after completion. Its strength is composition: transform a result, start a dependent operation, combine independent results, recover from failure, and apply deadlines without manually coordinating callbacks. Its danger is hidden execution policy, accidental blocking, confusing exception stages, and background work that outlives the request that created it.

A Stage Graph, Not Merely a Future

```
load customer ----\
                  +--> combine --> price --> response
load orders -----/
```

Each method creates a new stage. The original future is not modified.

```
CompletableFuture<Customer> customer =
    CompletableFuture.supplyAsync(() -> customerClient.load(id), ioExecutor);

CompletableFuture<CustomerView> view =
    customer.thenApply(this::toView);
```

CompletableFuture is also a Future, so join and get can block. Good composition delays blocking until a clearly chosen boundary or returns the stage to a framework that understands asynchronous completion.

Execution Policy: Async Is Not One Thing

Non-Async continuation methods such as thenApply may run in the thread that completes the previous stage, or immediately in the caller if already complete. Async variants schedule through an executor.

```
future.thenApply(this::transform);
future.thenApplyAsync(this::transform, cpuExecutor);
```

Async overloads without an executor generally use the ForkJoin common pool. That global pool is shared by unrelated work and is primarily designed for CPU-oriented fork/join tasks. Blocking database or HTTP calls there can consume its workers and interfere with other components.

Pass an explicitly owned executor when workload isolation, blocking behavior, context, or operational control matters.

thenApply Versus thenCompose

thenApply transforms a value synchronously and wraps the transformed result in a stage.

```
CompletableFuture<String> name = customer.thenApply(Customer::name);
```

If the transformation itself returns a CompletionStage, thenApply creates nesting:

```
CompletableFuture<CompletableFuture<Orders>>> nested =  
    customer.thenApply(c -> loadOrdersAsync(c.id()));
```

thenCompose flattens the dependent asynchronous operation:

```
CompletableFuture<Orders> orders =  
    customer.thenCompose(c -> loadOrdersAsync(c.id()));
```

The conceptual distinction matches map versus flatMap: thenApply transforms T to U; thenCompose chains T to CompletionStage<U>.

Combining Independent Work

thenCombine joins two independent results without blocking a worker merely to call get.

```
CompletableFuture<Customer> customer = loadCustomer(id);  
CompletableFuture<List<Order>> orders = loadOrders(id);  
  
CompletableFuture<CustomerPage> page = customer.thenCombine(  
    orders,  
    CustomerPage::new  
);
```

Start independent operations before combining them. Writing loadCustomer(id).thenCompose(c -> loadOrders(id)) serializes calls that could have overlapped.

Whether parallel dependency calls improve latency depends on downstream capacity and failure behavior. Fan-out increases concurrent load and can amplify a slow dependency.

allOf and Typed Result Collection

allOf completes when every supplied future completes, but its result type is Void. The original futures still hold typed results.

```
List<CompletableFuture<Item>> futures = ids.stream()  
    .map(this::loadItem)  
    .toList();  
  
CompletableFuture<List<Item>> all = CompletableFuture  
    .allOf(futures.toArray(CompletableFuture[]::new))  
    .thenApply(ignored -> futures.stream()  
        .map(CompletableFuture::join)  
        .toList());
```

The joins occur after allOf succeeds, so they do not normally wait. If one stage fails, allOf completes exceptionally, but other operations are not automatically cancelled. Define whether to collect partial results, fail fast, cancel siblings, or wait for all diagnostics.

Failure Is Part of the Graph

Exceptions thrown by a supplier or continuation complete that stage exceptionally. Dependent success-only stages are skipped until a recovery or observation stage handles the failure.

```
CompletableFuture<Price> price = loadPrice(id)  
    .exceptionally(error -> fallbackPrice(error));
```

exceptionally recovers only from failure and produces a replacement value. **handle** receives either result or error and always transforms. **whenComplete** observes success or failure but normally preserves the

original outcome unless its own action fails.

```
future.whenComplete((result, error) -> {  
    if (error != null) metrics.recordFailure(error);  
});
```

CompletionException and ExecutionException wrap underlying causes at join and get boundaries. Preserve the root cause and domain meaning when translating failures.

Recovery should be selective. Returning an empty list for authentication failure, data corruption, and a harmless missing recommendation treats unlike failures as equal and can create false success.

Timeouts and Deadlines

`orTimeout` completes a future exceptionally after the timeout. `completeOnTimeout` supplies a fallback value.

```
CompletableFuture<Quote> quote = loadQuote(id)  
    .orTimeout(800, TimeUnit.MILLISECONDS);
```

Completing the `CompletableFuture` exceptionally does not guarantee the underlying HTTP call, JDBC statement, or arbitrary supplier stops. Configure timeouts in the dependency client and propagate cancellation where supported.

An overall request deadline should be divided across children. Independent fixed timeouts at every layer can exceed the caller's remaining budget. Capture remaining time and refuse work that cannot finish meaningfully.

Cancellation Semantics

Cancelling a `CompletableFuture` marks it cancelled and causes dependent stages to fail accordingly, but it does not offer the same guaranteed interrupt attempt developers may expect from an executor-backed Future. The underlying action needs an explicit cancellation mechanism if stopping it matters.

Cancellation is a graph design question:

```
parent no longer needs result  
|  
+--> mark stage cancelled  
+--> cancel HTTP/JDBC operation if supported  
+--> prevent later side effects  
+--> release permits and context
```

Never assume a client timeout means background work cannot commit a write later. Use idempotency and transaction design for side effects.

Avoid Blocking Inside Stages

This pattern consumes an executor worker while waiting for work that could have been composed:

```
thenApply(value -> anotherFuture(value).join()) // hidden blocking
```

Use `thenCompose`. Blocking can be reasonable at a boundary such as a command-line main method, but blocking within a small shared pool can cause starvation or deadlock.

Also avoid performing long CPU work in a completion thread that should quickly release I/O resources. Move significant CPU transformation to an appropriate bounded executor with an `Async` method.

Fan-Out Needs a Bound

Creating one future for each of a million records starts a million logical operations and retains a large graph.

```
List<CompletableFuture<Result>> results = ids.stream()
    .map(this::callRemoteAsync)
    .toList();
```

Executor size may limit active threads, but the queue and future graph can still be enormous. Process bounded batches, use a Semaphore around dependency calls, use a bounded executor queue, or adopt a streaming system with backpressure.

Context and Transactions

ThreadLocal logging, security, tracing, and transaction context do not automatically follow a stage onto another executor. Capture only the required immutable context or use framework-supported propagation.

A Spring transaction is normally thread-bound. Starting `supplyAsync` inside a transactional method does not automatically extend that transaction to the executor thread. Design each asynchronous unit with its own explicit transaction boundary or keep transactional work on the owning thread.

```
request thread transaction
|
+-- async executor thread: different thread, no automatic transaction
```

This is a frequent production source of `LazyInitializationException`, missing security identity, and incomplete tracing.

CompletableFuture and Virtual Threads

`CompletableFuture` is valuable for explicit dependency graphs and APIs already modeled as `CompletionStage`. Virtual threads make straightforward blocking composition scalable for many I/O-heavy request paths.

```
Customer customer = customerClient.load(id);
Orders orders = orderClient.load(id);
```

Sequential blocking code is simpler but does not overlap independent calls unless tasks are forked. `CompletableFuture` expresses overlap but increases graph and context complexity. Choose based on concurrency needs, ecosystem, error semantics, observability, and team maintainability rather than fashion.

Testing Asynchronous Graphs

Inject executors rather than hiding global pools. Unit tests can use a direct executor for deterministic simple stages, while integration tests exercise real concurrency. Test success, each dependency failure, timeout, cancellation, executor rejection, partial completion, context propagation, and shutdown.

Avoid arbitrary sleeps. Control completion with test futures, latches, fake clocks where applicable, and bounded await assertions. A test that passes only because a machine happened to schedule quickly is not reliable.

Completion Is a Graph Event

A `CompletableFuture` pipeline is a directed graph of completion dependencies. Registering a continuation normally creates a dependent stage that becomes eligible when its inputs complete; it does not allocate one background thread that flows through the chain.

```

            inventory ----+
                          +--> combine --> price response
request --> customer -----+
      |
      +--> recommendations --> optional enrichment

```

Normal completion carries a value; exceptional completion carries a failure; cancellation is represented as exceptional completion. Each dependent stage has a rule for which input events enable it and how it transforms the outcome. This event model is more accurate than imagining a sequence of threads.

The graph reveals accidental serialization. If recommendations are independent of customer lookup but are created inside the customer continuation, they cannot start until customer lookup finishes. Starting both futures first and combining them preserves concurrency. Conversely, eager work wastes capacity when it is genuinely conditional. Shape the graph from data dependencies.

Non-Async and Async Continuations

A non-`Async` continuation such as `thenApply` may run in the thread that completes its prerequisite. If the prerequisite is already complete when attached, the attaching thread may execute it. Small transformations can run efficiently inline, but blocking or expensive code can unexpectedly occupy a network callback, scheduler, or request thread.

An `Async` method without an explicit executor normally uses `ForkJoinPool.commonPool()` under the default policy. That pool is shared across unrelated code and libraries. Sending blocking JDBC or slow remote calls there creates interference. The suffix does not mean "a new thread" and does not select a resource suited to the dependency.

```

CompletableFuture<Customer> customer =
    CompletableFuture.supplyAsync(() -> customerClient.load(id), ioExecutor);

CompletableFuture<CustomerView> view = customer.thenApplyAsync(
    this::renderCustomer,
    cpuExecutor);

```

Explicit executors document resource ownership and provide bulkheads. Use them selectively: forcing every tiny continuation through a queue adds latency. Ask which component may execute the function and whether it can block.

Flattening Preserves the Domain Type

`thenApply` maps `T` to an ordinary `U`. If the function already returns a future, `thenApply` produces a nested future. `thenCompose` flattens that asynchronous dependency.

```

thenApply(asyncFunction): Future<T> -> Future<Future<U>>
thenCompose(asyncFunction): Future<T> -> Future<U>

```

```

CompletableFuture<Order> order = loadCustomer(customerId)
    .thenCompose(customer -> createOrder(customer, request));

```

The nested form is not merely ugly. Its outer future can complete after producing the inner future while the real operation remains unfinished, confusing failure, cancellation, and lifecycle ownership. `thenCompose` states that the next asynchronous operation belongs to the same logical chain.

Combining, Racing, and Failure Policy

Use `thenCombine` when two independent successful values are both required. `thenAcceptBoth` is the side-effecting form when no result is returned. Either-style operations express a race between alternatives, but a naive "first completion always wins" summary is too broad once failures are involved. For hedged requests, define whether one failure waits for another possible success, how losing work is cancelled, and whether duplication is safe.

```
CompletableFuture<Quote> quote = priceFuture.thenCombine(
    discountFuture,
    (price, discount) -> new Quote(price.minus(discount)));
```

If either required input is exceptional, the combined computation cannot produce its normal result. The other operation may still continue because completion dependency is not cancellation ownership. Retain handles for expensive children and implement a deliberate deadline or cancellation policy.

`allOf` returns `CompletableFuture<Void>` because heterogeneous stages cannot form one statically typed list. For homogeneous work, retain the originals and collect after the gate:

```
List<CompletableFuture<Item>> calls = ids.stream()
    .map(id -> CompletableFuture.supplyAsync(() -> load(id), ioExecutor))
    .toList();

CompletableFuture<List<Item>> result = CompletableFuture
    .allOf(calls.toArray(CompletableFuture[]::new))
    .thenApply(ignored -> calls.stream()
        .map(CompletableFuture::join)
        .toList());
```

The joins are nonblocking after normal `allOf` completion. If an input fails, `allOf` is exceptional. Product semantics still must choose fail-all, successful items plus errors, a valid fallback, or selective retry. `allOf` coordinates completion; it does not decide partial-result policy.

Failure Operators Have Different Jobs

<code>exceptionally</code>	failure -> fallback value; success passes through
<code>exceptionallyCompose</code>	failure -> asynchronous recovery stage
<code>handle</code>	success or failure -> transformed outcome
<code>whenComplete</code>	observe success/failure; normally preserve outcome

`handle` always runs and can convert either path. `whenComplete` is intended for observation such as metrics or cleanup; it receives the value and failure but normally preserves the original outcome. Telemetry code should not throw casually and replace a business outcome.

Broad fallbacks can make an outage look like valid empty data. Unwrap and classify failures deliberately. Completion code often sees `CompletionException` or `ExecutionException` around the domain cause. Preserve distinctions between timeout, cancellation, validation, dependency unavailability, and defects.

At a synchronous boundary, `join()` throws unchecked `CompletionException`. `get()` follows `Future` and exposes checked `InterruptedException` and `ExecutionException`; its timed form also throws `TimeoutException`. Use `get` when interruptible waiting is part of the boundary. Use `join` only when unchecked translation is intentional or completion is already guaranteed.

Timeout Is Completion Policy, Not Work Termination

`orTimeout` exceptionally completes an incomplete future with `TimeoutException`. `completeOnTimeout` supplies a normal fallback. Both alter what observers see; neither proves underlying I/O stopped.

```
caller deadline expires ----> future completes with timeout
                             |
underlying socket/JDBC call -----+----- may still be running
```

If timed-out work continues, retries can fill pools after callers leave. Configure timeouts in the actual client, pass a remaining deadline through layers, and cancel child work where cancellation is meaningful. One end-to-end budget is safer than giving every sequential dependency the full original timeout.

Fallbacks must be domain-correct. Cached recommendations might be acceptable; "payment approved" is not. Distinguish timeout from executor rejection so operators can tell whether work waited too long or was never admitted.

Cancellation Is Exceptional Completion

For `CompletableFuture`, `cancel` is exceptional completion with `CancellationException`. One of normal completion, exceptional completion, or cancellation wins; later ordinary completion attempts do not replace it.

A crucial API fact is that `cancel(boolean mayInterruptIfRunning)` does not use that boolean to interrupt computation, because `CompletableFuture` does not generally control the executing thread. Cancelling a dependent stage does not automatically cancel prerequisites, and cancelling a parent does not imply domain-aware termination of every child. The graph expresses value dependency, not structured lifetime ownership.

To stop underlying activity, integrate with a client cancellation token, retain and cancel the actual submitted `Future`, close a relevant resource where safe, or make the task observe a cooperative signal. Test races among completion, timeout, and cancellation. Do not claim that `mayInterruptIfRunning=true` makes a `CompletableFuture` task stop.

Bounded Fan-Out and Failure Aggregation

Creating 50,000 futures is easy; executing 50,000 remote calls safely is not. A bounded executor limits queued platform-thread work, while a semaphore or client connection pool can limit active calls. With virtual threads, the semaphore remains important because cheap threads do not make a dependency unlimited.

```
return CompletableFuture.supplyAsync(() -> {
    boolean acquired = permits.tryAcquire(remainingMillis(), MILLISECONDS);
    if (!acquired) throw new CompletionException(new BusyException());
    try {
        return client.load(id);
    } finally {
        permits.release();
    }
}, ioExecutor);
```

For batch APIs, record failure by item rather than allowing one exception to erase information when partial results are useful. For fail-fast behavior, define ownership of remaining children. For collect-all, bound total wait by a deadline and preserve error classifications. Concurrency control and result semantics are separate decisions.

Completion Control and API Encapsulation

`CompletableFuture` is both a stage and a mutable completion object. Exposing it directly lets callers attempt `complete`, exceptional completion, or cancellation on an operation they may not own. Returning `CompletionStage` narrows the API; `minimalCompletionStage()` or `copy()` can strengthen separation where appropriate.

Forceful methods such as `obtrudeValue` and `obtrudeException` change what later observations see even after completion. Dependents that already observed the old result may disagree with later observers. These methods suit exceptional recovery or test infrastructure, not ordinary business flow.

Context, Transactions, and Virtual Threads

Stages may execute on different threads, so `ThreadLocal` logging, security, and tracing context does not automatically follow the request. Capture and restore supported context through framework instrumentation or executor wrappers. Avoid copying arbitrary thread locals, especially transaction state.

A Spring transaction is commonly thread-bound. Launching asynchronous work inside a transactional method does not include that work in the same atomic transaction. It may run after the original commits or rolls back. Pass stable identifiers or immutable data, and give asynchronous work its own transaction and idempotency semantics.

Virtual threads make sequential blocking calls readable and scalable. `CompletableFuture` remains useful for explicit fan-out, independent-result combination, callbacks, and async APIs. Do not replace a clear virtual-thread workflow with a graph merely to appear nonblocking. Choose the model that makes deadline, cancellation, ownership, and failure clearest.

A Strong Interview Explanation

`CompletableFuture` models a graph of completion dependencies, not a guaranteed background thread. Non-async stages may run on a completing or attaching thread; async stages without an executor normally use the common pool, so blocking work needs intentional resources. Use `thenApply` for synchronous mapping, `thenCompose` to flatten dependent async work, and combining operators for independent inputs.

Then describe the operational contract: classify failures rather than hiding them, propagate an end-to-end deadline, and remember that timeout or `CompletableFuture.cancel` changes completion state without necessarily stopping work. Bound fan-out to dependency capacity, propagate context deliberately, do not assume a transaction crosses threads, and prefer virtual threads when straightforward blocking control flow gives clearer ownership.

Production Perspective

Common incidents include blocking the common pool with JDBC, losing trace context, serializing supposedly parallel calls, unbounded fan-out, timeouts that leave work running, and broad exceptionally handlers turning outages into plausible but wrong data. Another frequent issue is a controller calling join, eliminating non-blocking benefits and tying up request threads.

Observe stage and dependency latency, executor active and queue metrics, rejection, timeout and cancellation, graph cardinality, common-pool utilization, downstream concurrency, and abandoned work after caller timeout.

Interview-Ready Summary

`CompletableFuture` models a value plus a graph of dependent stages. `thenApply` transforms a value, `thenCompose` flattens a dependent asynchronous operation, `thenCombine` combines independent results, and `allOf` coordinates many stages without producing typed results itself.

Non-Async and Async variants have different execution behavior. Avoid accidental use of the common pool for blocking work, hidden joins inside stages, and unbounded fan-out. Model failure, timeouts, cancellation, context, and transaction boundaries explicitly. `CompletableFuture` completion does not automatically stop underlying work.

Revision Notes

- `CompletableFuture` represents both completion and a stage graph.
- Every transformation returns a new stage.
- Non-Async continuations may run in the completing thread.
- Async overloads without an executor commonly use the common pool.
- Do not place uncontrolled blocking I/O on the common pool.
- `thenApply` maps T to U.
- `thenCompose` chains T to `CompletionStage<U>` and flattens it.
- `thenCombine` combines independent results.
- `allOf` returns `Void`; typed results remain in the original futures.
- `allOf` does not automatically cancel sibling work after failure.
- `exceptionally` recovers from failure.
- `handle` transforms either success or failure.
- `whenComplete` observes while normally preserving the outcome.
- Preserve root causes when unwrapping completion exceptions.
- Timeout of a stage does not guarantee underlying work stops.
- Cancellation requires cooperation from the actual operation.
- Avoid join inside stages when composition is possible.
- Bound fan-out, queues, and downstream concurrency.
- `ThreadLocal` and transaction context do not automatically cross executors.
- Virtual threads and `CompletableFuture` solve overlapping but different concerns.
- Test failure, timeout, cancellation, rejection, context, and shutdown deterministically.
- A stage graph expresses value dependencies, not structured task ownership.
- Independent futures must start independently to achieve concurrency.
- Non-async continuations may execute on a completing or attaching thread.
- `thenCompose` flattens a future returned by a continuation.
- `allOf` coordinates completion but does not define partial-result policy.
- `join` wraps failure unchecked; `get` exposes checked interruption and failure.
- `whenComplete` observes; `handle` transforms success or failure.
- Timeout methods do not prove underlying work stopped.
- `CompletableFuture` cancellation is exceptional completion.
- `mayInterruptIfRunning` has no effect for `CompletableFuture` cancellation.
- Bound fan-out independently from result aggregation semantics.
- Return a narrower stage view when callers must not control completion.
- Do not assume thread-bound transactions or context cross stage boundaries.